# Team ICB floorplanner: code, build and reproduction guide

ICCAD 2026 CAD Contest, Problem C (FloorSet-Lite) | 10/11/2026

## Latest results

| Measure (contest evaluator, 100 validation designs) | Team ICB 10/11 |
|---|---|
| Official weighted score (exp(blocks/12) weights) | 1.0465 |
| Ground-truth layouts, scored by the same evaluator | 1.108 |
| Published leader (10/6 leaderboard, raw cost) | 1.084 |
| Designs where we beat the ground truth's own score | 72 / 100 |
| Feasible layouts | 100 / 100 |
| Mean HPWL gap / area gap / relative violations | 0.022 / 0.036 / 0.011 |
| Mean runtime per design (8-core laptop, see 6) | 209 s |

Reference result file: iccad2026contest/official_eval_1011_pack200.json (every design's layout, cost and runtime). 1.0 means the ground truth's wirelength and area; the ground truth itself scores 1.108 because it breaks some of its own edge and cluster constraints.

This document is for someone who wants to reproduce and verify these results: what to check out, how to install and run it, how to compare their numbers with ours, and how every trained model was produced.

## 1. Get the code

```
git clone -b 'experiment1---(Claude-generated-version-of-Goldie-work)' \
    git@github.com:andyfox-rushc/FloorSet.git
cd FloorSet
# all solver commands below run from FloorSet/iccad2026contest
```

The repository is a fork of Intel's FloorSet repository. The top level holds Intel's dataset loaders and the 100 validation designs (LiteTensorDataTest/); our solver lives in iccad2026contest/ next to the contest evaluator. The solver is on the branch above only: the default branch (main) is Intel's code without it.

## 2. Install

No compilation step: everything is Python. Tested with Python 3.14.4 and the versions below (requirements.txt gives the minimum versions):

| Package | Tested version |
|---|---|
| torch (CPU build is enough) | 2.14.0 |
| numpy | 2.5.2 |
| shapely | 2.1.2 |
| matplotlib | 3.11.1 |
| tqdm | 4.70.0 |
| requests | 2.34.2 |

```
cd FloorSet
python3 -m venv venv
source venv/bin/activate
pip install -r iccad2026contest/requirements.txt
```

The scripts call ../venv/bin/python from iccad2026contest/, i.e. the venv inside FloorSet/. The validation data is in the repository; the 1M-design training set (about 15 GB download, 24 GB unpacked, into FloorSet/floorset_lite/) is only needed to retrain models (section 7) and downloads automatically on first use.

# 3. Run and verify

### Full run (the official number)

```
cd iccad2026contest
../venv/bin/python iccad2026_evaluate.py --evaluate my_optimizer.py \
    --output my_run.json            # ~6 h on 8 cores
../venv/bin/python scripts/compare_with_gt.py my_run.json official_eval_1011_pack200.json
```

The evaluator prints the total score; compare_with_gt.py scores the ground-truth layouts with the same evaluator and prints, for each result file, the weighted score, the mean gaps, how many designs beat the ground truth and the largest losses. All defaults are the configuration that produced 1.0465; no environment variables are needed.

### Quick check (about 10-15 minutes)

```
../venv/bin/python scripts/solver_ab.py --ids 0 98 --out quick.json
../venv/bin/python scripts/anchor_results.py --test-ids 0 49 98 --out-dir anchors
```

solver_ab.py solves chosen designs one at a time exactly as the evaluator does. Ours: design 0 1.046 and design 98 1.044 in the official run; over three further runs 1.014-1.037 and 1.007-1.039 (ground truth 1.091 and 1.080). anchor_results.py also draws each layout next to the ground truth (PNG files).

### What to expect

Results vary from run to run: the packer searches for a fixed wall-clock time, so machine speed and load change how far it gets. Our repeated runs of one configuration differ by about +-0.01 on the weighted score, and single designs much more (design 49: 1.159-1.274 over three runs). Run on an otherwise idle machine; a slower or busy machine scores worse. Six worker processes are used (FLOORSET_PROCS, default min(6, cpus)).

# 4. Files needed

Paths relative to FloorSet/. 'Intel' = unchanged from the upstream repository.

### Solver (run time)

| File | Role |
|---|---|
| iccad2026contest/my_optimizer.py | Entry point (MyOptimizer.solve), the whole pipeline |
| iccad2026contest/iccad2026_evaluate.py | Contest evaluator (Intel, unchanged) |
| iccad2026contest/rl/__init__.py | Package marker |
| iccad2026contest/rl/data.py | Contest inputs -> FloorplanInstance |
| iccad2026contest/rl/ordering.py | Placement order for the RL policy |
| iccad2026contest/rl/env.py | Grid placement environment (hard constraints) |
| iccad2026contest/rl/encoder.py | Graph network over the netlist |
| iccad2026contest/rl/networks.py | RL policy / value networks |
| iccad2026contest/rl/ppo.py | PPO rollouts and updates |
| iccad2026contest/rl/reward.py | Contest cost as the RL reward |
| iccad2026contest/rl/finetune.py | Per-design PPO fine-tuning of the policy |
| iccad2026contest/rl/checkpoint_meta.py | Checkpoint metadata (canvas utilization) |
| iccad2026contest/rl/compaction.py | Gravity compaction |
| iccad2026contest/rl/anneal.py | Annealing polish |
| iccad2026contest/rl/repack.py | Re-packing polish, exact cost helpers |
| iccad2026contest/rl/slicing.py | Slicing polish |
| iccad2026contest/rl/legalize.py | Wall repair / legalization |
| iccad2026contest/rl/analytic.py | Analytic placers, learned candidates, best-of |
| iccad2026contest/rl/learned_placer.py | Learned placer network (centres, relations) |
| iccad2026contest/rl/sizing.py | Differentiable shape fitting |
| iccad2026contest/rl/packer.py | Sequence-pair packer, squeeze, wirelength relaxation |
| cost.py utils.py visualize.py | Intel helpers imported by the evaluator |
| liteLoader.py lite_dataset.py litetestLoader.py | Intel data loaders |
| lite_dataset_test.py prime_dataset.py | Intel data loaders |
| LiteTensorDataTest/ (200 files) | Intel: the 100 validation designs + labels |

### Trained models (iccad2026contest/checkpoints/)

| File | Size | Role |
|---|---|---|
| policy.pt + policy.pt.meta.json | 304 KB | RL placement policy (PPO), canvas utilization 0.70 |
| placer_a.pt | 1.3 MB | Learned placer A: block centres + aspect ratios |
| placer_r.pt | 7.4 MB | Relation network 1 (pairwise relations + centres) |
| placer_relB.pt | 7.4 MB | Relation network 2 (independent training run) |

## Training, verification and documentation

| File | Role |
|---|---|
| iccad2026contest/rl/train.py, rl/pretrain.py | RL policy training (python -m rl.train) |
| iccad2026contest/scripts/train_placer.py | Learned placer / relation network training |
| iccad2026contest/scripts/polish_bench.py | Shared evaluator setup used by the scripts |
| iccad2026contest/scripts/__init__.py | Package marker |
| iccad2026contest/scripts/compare_with_gt.py | Verify a result file against the ground truth |
| iccad2026contest/scripts/solver_ab.py | Solve chosen designs one at a time |
| iccad2026contest/scripts/anchor_results.py | Solve + draw layouts vs ground truth |
| iccad2026contest/scripts/generate_code_doc_pdf.py | Builds this document |
| iccad2026contest/official_eval_1011_pack200.json | Reference result (1.0465) |
| iccad2026contest/algorithm.md | Design notes and every experiment, dated |
| iccad2026contest/requirements.txt | Python packages |

We checked this list by copying only these files into an empty directory and running the solver, the placer training and the policy training from there.

# 5. How the solver works

For each design my_optimizer.py builds candidate layouts, packs the most promising ones with a long search, and returns the cheapest by the exact contest cost:

| Stage | What it does |
|---|---|
| 1. RL policy | policy.pt places blocks one at a time on a grid (GNN encoder + CNN position head), fine-tuned on the design with PPO for a few seconds ('base') |
| 2. Analytic placers | Five wirelength-driven global placements from scratch at canvas utilizations 0.75-1.0 ('scratch...' candidates) |
| 3. Learned placers | Networks trained on 1M ground-truth layouts predict block positions and pairwise left/right/below/above relations ('learned1-4') |
| 4. Shape fitting | Gradient descent on soft-block aspect ratios for a fixed arrangement |
| 5. Best-of | All candidates scored; the best plus the four learned ones go on |
| 6. Packer | Sequence-pair annealing on the exact contest cost, 200 s at 77 blocks (scaled by (blocks/77)^2), one process per layout, shape refit halfway |
| 7. Finish | Squeeze, wirelength relaxation, then the cheapest layout inside the box where the pins sit (preferred within 5% of the best cost) |

### The policy and models used

| Candidate | Checkpoint | Arrangement from |
|---|---|---|
| base | policy.pt | RL policy |
| scratch0.75 ... scratch1.0 | (none) | analytic placement |
| learned | placer_a.pt | predicted centres |
| learned2 | placer_r.pt | predicted pairwise relations |
| learned3 | placer_r.pt | predicted centres |
| learned4 | placer_relB.pt | predicted pairwise relations |

### Settings (defaults = the 1.0465 run)

| Environment variable | Default | Meaning |
|---|---|---|
| FLOORSET_PACK | 200 | Packer seconds per layout at 77 blocks |
| FLOORSET_PROCS | min(6, cpus) | Worker processes |
| FLOORSET_CANDIDATES | base,scratch0.75..1.0,learned..learned4 | Candidate layouts |
| FLOORSET_CHECKPOINT | checkpoints/policy.pt | RL policy |
| FLOORSET_PLACER, ..._PLACER4 | see table above | Learned placer checkpoints |

algorithm.md documents every other switch, with the measurement that set its default.

# 6. Machine used for the results

Intel Core Ultra 7 258V (8 cores), 16 GB RAM, Windows 11 + WSL2 (Linux 6.18), CPU only. One official run of the 100 designs took about 5.6 hours (209 s per design on average, most of it the packer's fixed time budget).

# 7. Retraining the models (optional)

The checkpoints in the repository are the ones that produced the results. To retrain them, run the commands below from iccad2026contest/. Training stops on a wall-clock limit and runs on several threads, so a retrained model will not be bit-identical; expect the solver's score to land within the run-to-run noise rather than exactly. Judge a retrained model by a full evaluation run (section 3), not by its validation error: training network 1 on to 358k designs lowered that error but scored worse (1.0709).

### Training data

```
../venv/bin/python -c "from iccad2026_evaluate import get_training_dataloader; \
    get_training_dataloader(data_path='../', batch_size=1)"   # -> ../floorset_lite/
```

### RL policy: checkpoints/policy.pt

```
../venv/bin/python -m rl.train --iterations 4000 --episodes-per-iter 4 \
    --ppo-epochs 4 --checkpoint checkpoints/policy.pt --checkpoint-every 500 \
    --threads 3 --data-seed 12345
```

From scratch: reward-prediction pretraining of the encoder, then PPO over the training designs with an adaptive canvas utilization (settles at 0.70). We use the iteration-4000 checkpoint (13.4 hours on 3 threads); longer training did not improve the pipeline.

### Learned placer A: checkpoints/placer_a.pt (128 wide, 6 layers)

```
../venv/bin/python scripts/train_placer.py --hours 7 --threads 2 --val-every 10000 \
    --out checkpoints/placer.pt
```

placer_a.pt is the snapshot after about 290k training designs (validation centre error 0.067), taken while the run continued; to match it, stop near 290k designs.

### Base for the relation networks: placer_d.pt (256 wide, 8 layers)

```
../venv/bin/python scripts/train_placer.py --hours 6 --threads 4 --seed 4 --lr 5e-4 \
    --hidden 256 --layers 8 --norm --val-every 5000 --out checkpoints/placer_d.pt
```

### Relation network 1: checkpoints/placer_r.pt (two stages)

```
../venv/bin/python scripts/train_placer.py --hours 5 --threads 4 --seed 5 --lr 3e-4 \
    --hidden 256 --layers 8 --norm --rel --resume checkpoints/placer_d.pt \
    --val-every 5000 --out checkpoints/placer_rel.pt
../venv/bin/python scripts/train_placer.py --hours 6 --threads 6 --seed 6 --lr 2e-4 \
    --hidden 256 --layers 8 --norm --rel --resume checkpoints/placer_rel.pt \
    --val-every 5000 --out checkpoints/placer_r.pt
```

About 312k designs in total; validation centre error 0.050, relation accuracy 0.918.

### Relation network 2: checkpoints/placer_relB.pt (independent of network 1)

```
../venv/bin/python scripts/train_placer.py --hours 8 --threads 4 --seed 31 --lr 2e-4 \
    --hidden 256 --layers 8 --norm --rel --resume checkpoints/placer_d.pt \
    --val-every 5000 --out checkpoints/placer_relB.pt
```

About 357k designs; validation centre error 0.050, relation accuracy 0.917.